

TP4 – NoSQL Injection & Requêtes Sûres

Module : **Cybersécurité NoSQL**

Hasna Daoui

Date : 19 mai 2026

Table des matières

1	Introduction	3
1.1	Contexte pédagogique	3
1.2	Objectifs	3
2	Présentation des Technologies Utilisées	3
2.1	MongoDB	3
2.2	Node.js et Express	3
2.3	Docker	4
3	Description des Travaux Pratiques	4
3.1	Phase 1 : Mise en place de l'environnement	4
3.2	Phase 2 : Déploiement de l'API vulnérable	4
3.3	Phase 3 : Démonstration contrôlée des attaques	4
3.4	Phase 4 : Correction et sécurisation	4
3.5	Phase 5 : Tests de régression	4
3.6	Phase 6 : Endpoint sécurisé supplémentaire	4
4	Architecture de l'Environnement	5
4.1	Diagramme de l'architecture	5
4.2	Structure du projet	5
5	Mesures de Sécurité Appliquées	5
5.1	Le pattern safeFilter	5
5.2	Validation de type	5
5.3	Filtrage des opérateurs \$	6
5.4	Projection minimale	6
5.5	Prévention du Mass Assignment	6
6	Vulnérabilités Étudiées	6
6.1	Injection NoSQL vs Injection SQL	6
6.2	Vecteur 1 : Contournement d'authentification (<i>ne</i>)	6
6.3	Vecteur 2 : Exfiltration de données (<i>regex</i>)	6
6.4	Vecteur 3 : Mass Assignment	7
7	Captures d'Écran	7
7.1	Phase 1 : Environnement et données	7
7.2	Phase 2 : API Vulnérable	10
7.3	Phase 3 : Attaques	12
7.4	Phase 4 : Serveur Sécurisé	12
7.5	Phase 5 : Tests de sécurité supplémentaires	13
7.6	Phase 6 : Fonction hasNoMongoOperators	14
7.7	Phase 7 : Endpoint /users/update-email	15

8	Difficultés Rencontrées	15
8.1	Problème 1 : Conteneur MongoDB inexistant	16
8.2	Problème 2 : Données absentes de la base	16
8.3	Problème 3 : Commandes curl mal formées	16
8.4	Problème 4 : Sortie de mongosh	16
9	Conclusion	16
9.1	Checklist de sécurité retenue	16

1 Introduction

Ce TP aborde la sécurité applicative dans les bases de données NoSQL, plus précisément MongoDB. Contrairement aux injections SQL classiques qui exploitent la concaténation de chaînes dans des requêtes textuelles, les injections NoSQL exploitent la nature JSON des filtres MongoDB pour modifier la logique des requêtes sans casser de syntaxe.

1.1 Contexte pédagogique

Les trois premières séances ont posé les bases opérationnelles : installation de MongoDB sous Docker, opérations CRUD avec `mongosh`, modélisation par embedding et référence, requêtes d'agrégation, indexation sur la base `soc_lite`. La Séance 3 a introduit le hardening infrastructure (SCRAM, RBAC, bindIp). Ce TP4 complète ces connaissances par la sécurité applicative.

1.2 Objectifs

1. Expliquer ce qu'est une injection NoSQL et en quoi elle diffère de l'injection SQL
2. Identifier les points de contact où une entrée utilisateur peut contaminer un filtre MongoDB
3. Reproduire localement deux types d'injections NoSQL documentées
4. Appliquer le pattern `safeFilter` pour construire des requêtes sécurisées
5. Configurer une projection minimale (principe du moindre privilège sur les données)

2 Présentation des Technologies Utilisées

2.1 MongoDB

MongoDB est une base de données NoSQL orientée documents stockant les données au format BSON (Binary JSON). Les requêtes utilisent des filtres JSON enrichis d'opérateurs spéciaux préfixés par `$`.

TABLE 1 – Opérateurs MongoDB exploités dans les attaques

Opérateur	Description	Usage malveillant
<code>\$ne</code>	Not Equal	Contournement d'authentification
<code>\$regex</code>	Expression régulière	Exfiltration de données
<code>\$gt</code>	Greater Than	Sélection étendue
<code>\$where</code>	JavaScript conditionnel	Exécution de code

2.2 Node.js et Express

Node.js est le runtime JavaScript côté serveur. Express est le framework web minimaliste utilisé pour construire l'API REST intermédiaire entre le client et MongoDB.

2.3 Docker

Docker permet de conteneuriser MongoDB pour un environnement local reproductible et isolé.

```
1 docker run -d --name mongo-soc -p 27017:27017 \\  
2 -e MONGO_INITDB_ROOT_USERNAME=admin \\  
3 -e MONGO_INITDB_ROOT_PASSWORD=secret \\  
4 mongo:latest
```

Listing 1 – Commande de déploiement MongoDB

3 Description des Travaux Pratiques

3.1 Phase 1 : Mise en place de l'environnement

Déploiement du conteneur Docker MongoDB, création de la base `soc_lite`, insertion des données de test (utilisateurs et logs de sécurité).

3.2 Phase 2 : Déploiement de l'API vulnérable

Construction d'une API Express avec deux endpoints vulnérables :

- POST `/logs/search` – passe `req.body` directement dans `find()`
- POST `/auth/login` – passe `req.body.password` directement dans `findOne()`

3.3 Phase 3 : Démonstration contrôlée des attaques

Reproduction de deux vecteurs d'attaque :

1. **Contournement d'authentification** via `$ne`
2. **Exfiltration de données** via `$regex`

3.4 Phase 4 : Correction et sécurisation

Implémentation du pattern `safeFilter` avec :

- Validation de type (`typeof`)
- Validation de format (regex IPv4)
- Allow-list de valeurs (`Array.includes`)
- Projection minimale
- Comparaison mot de passe côté JavaScript

3.5 Phase 5 : Tests de régression

Vérification que les attaques sont bloquées et que les requêtes légitimes fonctionnent toujours.

3.6 Phase 6 : Endpoint sécurisé supplémentaire

Implémentation de `/users/update-email` avec protection contre le *mass assignment*.

4 Architecture de l'Environnement

4.1 Diagramme de l'architecture

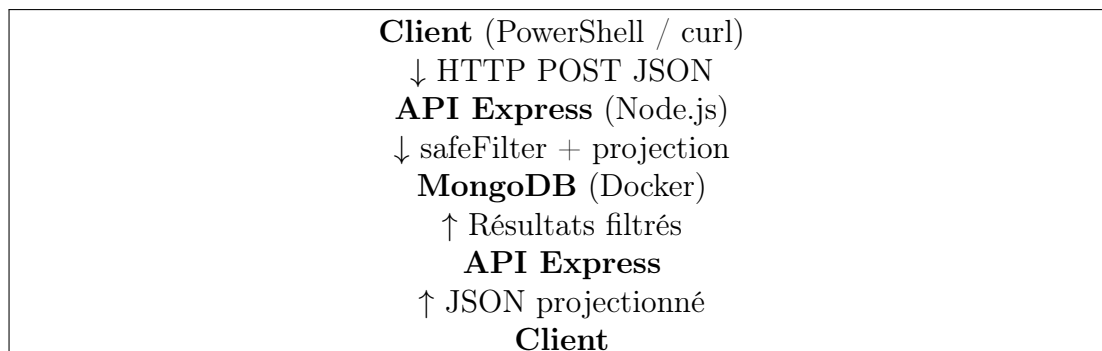


FIGURE 1 – Flux de données sécurisé

4.2 Structure du projet

```

TP4/
code/
  server-vulnerable.js    # API avec vuln\erabilit\es
  server-safe.js          # API corrig\ee
  package.json
screenshots/              # Captures d'\ecran
report/
  rapport-tp4.tex         # Ce document
  
```

5 Mesures de Sécurité Appliqu

5.1 Le pattern safeFilter

Le principe fondamental est de **ne jamais passer req.body directement à find()**. Chaque champ est extrait, validé, puis copié dans un objet construit par l'application.

```

1 const safeFilter = {};
2
3 if (req.body.src_ip !== undefined) {
4   if (!isValidIPv4(req.body.src_ip))
5     return res.status(400).json({ error: 'src_ip invalide' });
6   safeFilter.src_ip = req.body.src_ip; // Champ valid\e uniquement
7 }
8 // Les champs non pr\evus sont silencieusement ignor\es
  
```

Listing 2 – Pattern safeFilter

5.2 Validation de type

```

1 function isCleanString(val) {
2   if (typeof val !== 'string') return false; // Rejette objects
  
```

```

3  if (val.trim().length === 0) return false; // Rejette vide
4  if (val.length > 100) return false;        // Limite longueur
5  return true;
6 }

```

5.3 Filtrage des opérateurs \$

Toute clé commençant par \$ dans un objet utilisateur est une tentative d'injection d'opérateur MongoDB.

5.4 Projection minimale

```

1 .project({ _id: 0, ts: 1, src_ip: 1, action: 1, status: 1 })

```

Seuls les champs nécessaires sont retournés. Les champs sensibles (`password_hash`, `internal_note`) restent invisibles.

5.5 Prévention du Mass Assignment

Pour les mises à jour, l'application définit explicitement les champs modifiables :

```

1 const allowedFields = ['email', 'username'];
2 const hasExtraFields = Object.keys(req.body)
3   .some(field => !allowedFields.includes(field));
4 if (hasExtraFields) return res.status(400).json({ error: '...' });

```

6 Vulnérabilités Étudiées

6.1 Injection NoSQL vs Injection SQL

TABLE 2 – Comparaison SQL Injection vs NoSQL Injection

Aspect	SQL Injection	NoSQL Injection
Vecteur	Chaîne concaténée	Objet JSON inséré
Payload	' OR '1'='1	{"\$ne":""}
Mécanisme	Cassure syntaxique	Injection d'opérateurs JSON
Défense	Requêtes préparées	Validation + safeFilter

6.2 Vecteur 1 : Contournement d'authentification (*ne*)

Payload : {"username":"alice","password":{"\$ne":""}}

MongoDB interprète : “retourne Alice dont le mot de passe est différent de chaîne vide” – ce qui est vrai pour tout utilisateur ayant un mot de passe.

6.3 Vecteur 2 : Exfiltration de données (*regex*)

Payload : {"src_ip":{"\$regex":""}}

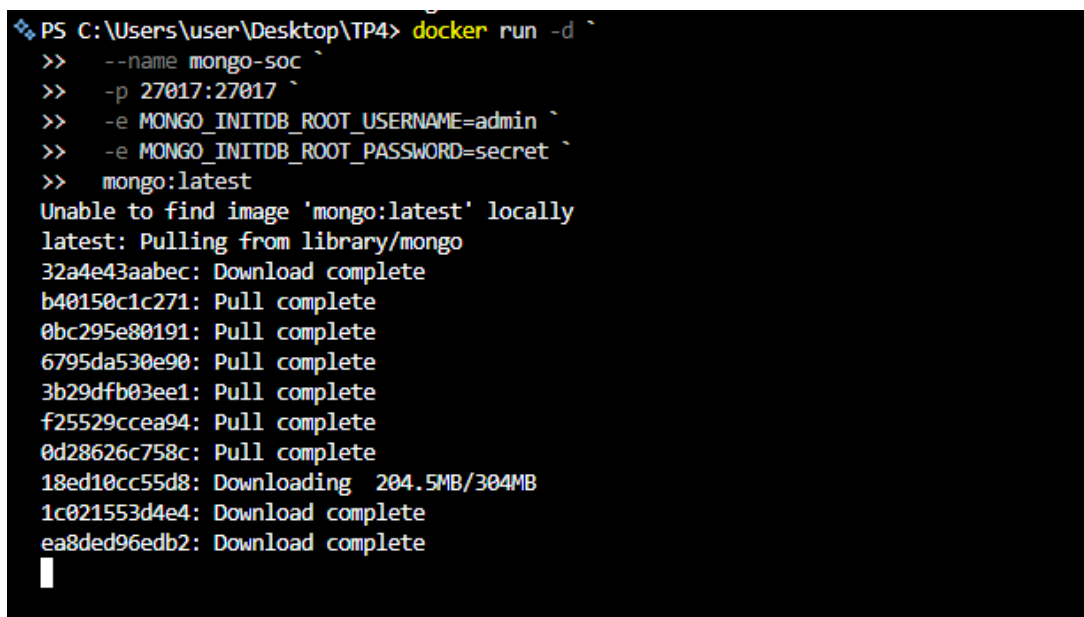
Une regex vide matche toute chaîne, retournant tous les documents de la collection.

6.4 Vecteur 3 : Mass Assignment

Injection de champs non autorisés (role, is_admin) lors d'une mise à jour apparemment légitime.

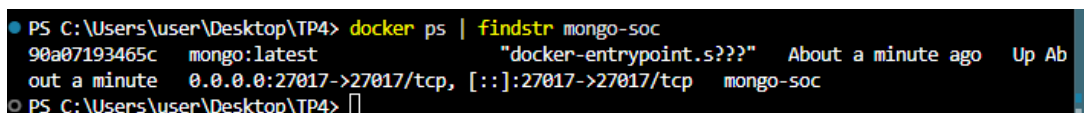
7 Captures d'Écran

7.1 Phase 1 : Environnement et données



```
PS C:\Users\user\Desktop\TP4> docker run -d `
>> --name mongo-soc `
>> -p 27017:27017 `
>> -e MONGO_INITDB_ROOT_USERNAME=admin `
>> -e MONGO_INITDB_ROOT_PASSWORD=secret `
>> mongo:latest
Unable to find image 'mongo:latest' locally
latest: Pulling from library/mongo
32a4e43aabec: Download complete
b40150c1c271: Pull complete
0bc295e80191: Pull complete
6795da530e90: Pull complete
3b29dfb03ee1: Pull complete
f25529ccea94: Pull complete
0d28626c758c: Pull complete
18ed10cc55d8: Downloading 204.5MB/304MB
1c021553d4e4: Download complete
ea8ded96edb2: Download complete
```

FIGURE 2 – Déploiement du conteneur MongoDB avec Docker



```
PS C:\Users\user\Desktop\TP4> docker ps | findstr mongo-soc
90a07193465c mongo:latest "docker-entrypoint.s???" About a minute ago Up Ab
out a minute 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp mongo-soc
PS C:\Users\user\Desktop\TP4>
```

FIGURE 3 – Vérification que le conteneur mongo-soc est actif


```

PS C:\Users\user\Desktop\TP4> docker exec -it mongo-soc mongosh -u admin -p secret --authenticationDatabase admin
Current Mongosh Log ID: 6a0ba3d3eacfbff4ad9df8a2
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.8.3
Using MongoDB:      8.2.9
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically
(https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
  The server generated these startup warnings when booting
  2026-05-18T23:39:52.331+00:00: Using the XFS filesystem is strongly recommended with the Wired
  Tiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  2026-05-18T23:39:53.618+00:00: For customers running the current memory allocator, we suggest
  changing the contents of the following sysfsFile
  2026-05-18T23:39:53.618+00:00: For customers running the current memory allocator, we suggest
  changing the contents of the following sysfsFile
  2026-05-18T23:39:53.618+00:00: We suggest setting the contents of sysfsFile to 0.
  2026-05-18T23:39:53.618+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause
  performance problems.
-----
test> 

```

FIGURE 4 – Connexion à MongoDB via mongosh

```

cause performance problems.
-----

test> show dbs
admin   100.00 KiB
config 108.00 KiB
local   72.00 KiB
test> use soc_lite
| show collections
| db.users.find()
| db.security_logs.find()
switched to db soc_lite
soc_lite> show collections

soc_lite> db.users.find()

soc_lite> db.security_logs.find()

soc_lite> // Create users collection with test data
| db.users.insertMany([
|   { username: "alice", password: "motdepasse123", role: "analyst" },
|   { username: "bob",   password: "secret456",    role: "admin" },
|   { username: "carol", password: "carol789",     role: "viewer" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0ba665eacfbff4ad9df8a3'),
    '1': ObjectId('6a0ba665eacfbff4ad9df8a4'),
    '2': ObjectId('6a0ba665eacfbff4ad9df8a5')
  }
}

```

FIGURE 5 – Création des utilisateurs de test dans la base soc_lite

```
soc_lite> show collections
security_logs
users
soc_lite> 
```

FIGURE 6 – Vérification des collections existantes

```
soc_lite> db.security_logs.insertMany([
|   { src_ip: "192.168.1.10", action: "login", status: "open", ts: new Date() },
|   { src_ip: "192.168.1.11", action: "logout", status: "closed", ts: new Date() },
|   { src_ip: "10.0.0.5", action: "alert", status: "investigating", ts: new Date() }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0ba66feacfbff4ad9df8a6'),
    '1': ObjectId('6a0ba66feacfbff4ad9df8a7'),
    '2': ObjectId('6a0ba66feacfbff4ad9df8a8')
  }
}
soc_lite> db.users.find({}, { username: 1, role: 1, password: 1, _id: 0 })
[
  { username: 'alice', password: 'motdepasse123', role: 'analyst' },
  { username: 'bob', password: 'secret456', role: 'admin' },
  { username: 'carol', password: 'carol789', role: 'viewer' }
]
soc_lite> db.security_logs.find()
[
  {
    _id: ObjectId('6a0ba66feacfbff4ad9df8a6'),
    src_ip: '192.168.1.10',
    action: 'login',
    status: 'open',
    ts: ISODate('2026-05-18T23:53:19.231Z')
  },
  {
    _id: ObjectId('6a0ba66feacfbff4ad9df8a7'),
```

FIGURE 7 – Insertion des logs de sécurité et vérification

7.2 Phase 2 : API Vulnérable

```
JS server.js > ...
1 const express = require('express');
2 const { MongoClient } = require('mongodb');
3 const app = express();
4 const PORT = 3000;
5 const URI = 'mongodb://admin:secret@localhost:27017/soc_lite?authSource=admin';
6 app.use(express.json());
7 let db;
8 MongoClient.connect(URI).then(client => {
9   db = client.db('soc_lite');
10  console.log('[OK] Connecte a MongoDB');
11  app.listen(PORT, () => console.log(`[OK] API sur :${PORT}`));
12 }).catch(err => { console.error(err); process.exit(1); });
13 // ENDPOINT 1: Search logs - VULNERABLE
14 app.post('/logs/search', async (req, res) => {
15   try {
16     const results = await db.collection('security_logs')
17       .find(req.body)
18       .limit(20).toArray();
19     res.json({ count: results.length, data: results });
20   } catch (err) { res.status(500).json({ error: err.message }); }
21 });
22 // ENDPOINT 2: Login - VULNERABLE
23 app.post('/auth/login', async (req, res) => {
24   try {
25     const { username, password } = req.body;
26     const user = await db.collection('users')
27       .findOne({ username, password });
28     if (user) res.json({ success: true, message: `Bienvenue ${user.username}`, role
```

FIGURE 8 – Code source de l'API vulnérable (server.js)

```
PS C:\Users\user\Desktop\TP4> node server.js
[OK] Connecte a MongoDB
[OK] API sur :3000

```

FIGURE 9 – Démarrage du serveur vulnérable

```

PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/auth/login" `
>> -ContentType "application/json" `
>> -Body '{"username":"alice","password":"motdepasse123"}'
Invoke-RestMethod : {"success":false,"message":"Identifiants incorrects"}
At line:1 char:1
+ Invoke-RestMethod `
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExc
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/logs/search" `
>> -ContentType "application/json" `
>> -Body '{"src_ip":"192.168.1.10"}'

count data
-----
0 {}

PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/auth/login" `
>> -ContentType "application/json" `
>> -Body '{"username":"alice","password":"WRONG"}'
Invoke-RestMethod : {"success":false,"message":"Identifiants incorrects"}
At line:1 char:1
+ Invoke-RestMethod `
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExc
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

```

FIGURE 10 – Test initial : login avec mauvais mot de passe (refusé)

```

PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/auth/login" `
>> -ContentType "application/json" `
>> -Body '{"username":"alice","password":"motdepasse123"}'

success message      role
-----
True Bienvenue alice analyst

PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/logs/search" `
>> -ContentType "application/json" `
>> -Body '{"src_ip":"192.168.1.10"}'

count data
-----
1 {@_id=6a0ba66feacfbbff4ad9df8a6; src_ip=192.168.1.10; action=login; status=open; ts=2026-05-18T23:53:19.231Z}}

PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/auth/login" `
>> -ContentType "application/json" `
>> -Body '{"username":"alice","password":"WRONG"}'
Invoke-RestMethod : {"success":false,"message":"Identifiants incorrects"}
At line:1 char:1
+ Invoke-RestMethod `
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\WINDOWS\system32>

```

FIGURE 11 – Test : login avec bons identifiants (succès)

```

PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/logs/search" `
>> -ContentType "application/json" `
>> -Body '{"src_ip":"192.168.1.10"}'

count data
-----
1 {@_id=6a0ba66feacfbbff4ad9df8a6; src_ip=192.168.1.10; action=login; status=open; ts=2026-05-18T23:53:19.231Z}}

PS C:\WINDOWS\system32>

```

FIGURE 12 – Recherche normale de logs par IP

7.3 Phase 3 : Attaques

```
PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/auth/login" `
>> -ContentType "application/json" `
>> -Body '{"username":"alice","password":{"$ne":""}}'

success message      role
-----
True Bienvenue alice analyst

PS C:\WINDOWS\system32> |
```

FIGURE 13 – **ATTAQUE 1** : Contournement d'authentification via `$ne` – Login réussi SANS mot de passe

```
PS C:\WINDOWS\system32> Invoke-RestMethod `
>> -Method POST `
>> -Uri "http://localhost:3000/logs/search" `
>> -ContentType "application/json" `
>> -Body '{"src_ip":{"$regex":""}}'

count data
3 [{_id=6a0ba66feacfbff4ad9df8a6; src_ip=192.168.1.10; action=login; status=open; ts=2026-05-18T23:53:19.231Z}, {_id=6a0ba66feacfbff4ad9df...

PS C:\WINDOWS\system32> |
```

FIGURE 14 – **ATTAQUE 2** : Exfiltration de tous les logs via `$regex` vide – count: 3

7.4 Phase 4 : Serveur Sécurisé

```
PS C:\Users\user\Desktop\TP4> node server-safe.js
[OK] Connecte a MongoDB
[OK] API securisee sur : 3000
|
```

FIGURE 15 – Démarrage du serveur sécurisé (server-safe.js)

```
PS C:\WINDOWS\system32> Get-Process node -ErrorAction SilentlyContinue
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
247	50	82420	18008	2.64	22400	1	node

```
PS C:\WINDOWS\system32>
```

FIGURE 16 – Vérification du processus Node.js en cours d'exécution

```
PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/auth/login" -ContentType "application/json" -Body '{"username":"alice","password":{"$ne":""}}'
Invoke-RestMethod : {"error":"username et password doivent etre des chaines"}
At line:1 char:1
+ Invoke-RestMethod -Method POST -Uri "http://localhost:3000/auth/login ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\WINDOWS\system32>
```

FIGURE 17 – DÉFENSE : L'attaque \$ne est bloquée (HTTP 400)

```
PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/logs/search" -ContentType "application/json" -Body '{"src_ip":{"$regex":""}}'
Invoke-RestMethod : {"error":"src_ip invalide - format IPv4 requis"}
At line:1 char:1
+ Invoke-RestMethod -Method POST -Uri "http://localhost:3000/logs/searc ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\WINDOWS\system32>
```

FIGURE 18 – DÉFENSE : L'attaque \$regex est bloquée (HTTP 400)

```
PS C:\WINDOWS\system32> # Login
PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/auth/login" -ContentType "application/json" -Body '{"username":"alice","password":"motdepasse123"}'
success message      role
-----
True Bienvenue alice analyst

PS C:\WINDOWS\system32> # Search
PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/logs/search" -ContentType "application/json" -Body '{"src_ip":"192.168.1.10"}'
count data
-----
1 [{"src_ip":192.168.1.10; action=login; status=open; ts=2026-05-18T23:53:19.231Z}]

PS C:\WINDOWS\system32>
```

FIGURE 19 – Les requêtes légitimes fonctionnent toujours normalement

7.5 Phase 5 : Tests de sécurité supplémentaires

```
PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/logs/search" -ContentType "application/json" -Body '{"action":{"$gt":""}}'
Invoke-RestMethod : {"error":"action invalide"}
At line:1 char:1
+ Invoke-RestMethod -Method POST -Uri "http://localhost:3000/logs/searc ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/logs/search" -ContentType "application/json" -Body '{"src_ip":"192.168.1.10","role":"admin"}'
count data
-----
1 [{"src_ip":192.168.1.10; action=login; status=open; ts=2026-05-18T23:53:19.231Z}]

PS C:\WINDOWS\system32>
```

FIGURE 20 – Attaque \$gt bloquée + champ non prévu ignoré

```
soc_lite> db.security_logs.updateOne(
|   { src_ip: "192.168.1.10" },
|   { $set: { password_hash: "HASH_FICTIF_SENSIBLE", internal_note: "ticket_17_confidentiel" } }
| )
| exit
[Function (anonymous)] { help: [Function (anonymous)] Help }
soc_lite> 
```

FIGURE 21 – Ajout de données sensibles (password_hash, internal_note) pour tester la projection

```
PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/logs/search" -ContentType "application/json" -Body '{"src_ip":"192.168.1.10"}'

count data
-----
1 [{src_ip=192.168.1.10; action=login; status=open; ts=2026-05-18T23:53:19.231Z}]

PS C:\WINDOWS\system32>
```

FIGURE 22 – Vérification : les champs sensibles sont absents de la réponse API

7.6 Phase 6 : Fonction hasNoMongoOperators

```
PS C:\Users\user\Desktop\TP4> node
Welcome to Node.js v22.19.0.
Type ".help" for more information.
> function hasNoMongoOperators(obj) {
...   if (typeof obj !== 'object' || obj === null) return true;
...   for (const key of Object.keys(obj)) {
...     if (key.startsWith('$')) return false;
...     if (!hasNoMongoOperators(obj[key])) return false;
...   }
...   return true;
... }
undefined
>
> hasNoMongoOperators({ username: "alice" })
true
> hasNoMongoOperators({ password: { $ne: "" } })
false
> hasNoMongoOperators({ a: { b: { $gt: 5 } } })
false
> hasNoMongoOperators("chaîne simple")
true
> hasNoMongoOperators([ { $where: "true" } ])
false
> 
```

FIGURE 23 – Tests de la fonction hasNoMongoOperators() dans Node.js REPL

7.7 Phase 7 : Endpoint /users/update-email

```

9
10 app.post('/users/update-email', async (req, res) => {
11   // 1. Type validation
12   if (typeof req.body.email !== 'string')
13     return res.status(400).json({ error: 'email doit etre une chaine' });
14
15   const email = req.body.email.trim();
16
17   // 2. Format validation
18   const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
19   if (!emailRegex.test(email))
20     return res.status(400).json({ error: 'format email invalide' });
21
22   // 3. Allow-list: reject extra fields (prevents mass assignment)
23   const allowedFields = ['email', 'username'];
24   const receivedFields = Object.keys(req.body);
25   const hasExtraFields = receivedFields.some(field => !allowedFields.includes(field));
26   if (hasExtraFields)
27     return res.status(400).json({ error: 'champs non autorises detectes' });
28
29   // 4. Username validation
30   if (typeof req.body.username !== 'string' || req.body.username.trim().length < 3)
31     return res.status(400).json({ error: 'username invalide' });
32
33   try {
34     // 5. Update ONLY email
35     const result = await db.collection('users').updateOne(

```

FIGURE 24 – Code du endpoint /users/update-email avec protection mass assignment

```

PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/users/update-email" -ContentType "application/json" -Body '{"user
name":"alice","email":"alice@securebank.com"}'
success message      email
-----
True Email mis a jour alice@securebank.com
PS C:\WINDOWS\system32>

```

FIGURE 25 – Mise à jour d'email valide – succès

```

PS C:\WINDOWS\system32> Invoke-RestMethod -Method POST -Uri "http://localhost:3000/users/update-email" -ContentType "application/json" -Body '{"user
name":"alice","email":"hacker@evil.com","role":"admin"}'
Invoke-RestMethod : {"error":"champs non autorises detectes"}
At line:1 char:1
+ Invoke-RestMethod -Method POST -Uri "http://localhost:3000/users/upda ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\WINDOWS\system32>

```

FIGURE 26 – DÉFENSE : Tentative de mass assignment bloquée (HTTP 400)

8 Difficultés Rencontrées

8.1 Problème 1 : Conteneur MongoDB inexistant

Symptôme : `Error response from daemon: No such container: mongo-soc`

Cause : Le conteneur n'avait jamais été créé.

Solution : Création du conteneur avec `docker run` au lieu de `docker start`.

8.2 Problème 2 : Données absentes de la base

Symptôme : Login refusé même avec bons identifiants, recherche retournant 0 résultats.

Cause : Les insertions dans `mongosh` n'ont pas été exécutées correctement (commandes collées sans exécution).

Solution : Recréation manuelle des collections et documents de test.

8.3 Problème 3 : Commandes curl mal formées

Symptôme : `SyntaxError: Expected property name or '' in JSON`

Cause : PowerShell interprète mal les guillemets dans les commandes `curl` multilignes.

Solution : Utilisation de `Invoke-RestMethod` avec syntaxe PowerShell native.

8.4 Problème 4 : Sortie de mongosh

Symptôme : La commande `exit` retourne `[Function (anonymous)]` au lieu de quitter.

Solution : Utilisation de `.exit` (avec le point) ou `Ctrl+D`.

9 Conclusion

Ce TP a démontré de manière concrète les mécanismes des injections NoSQL et leur impact potentiel sur une application web. Les démonstrations contrôlées ont montré que :

1. Les injections NoSQL sont **réelles et dangereuses**, même sans connaissance préalable du mot de passe
2. La différence fondamentale avec SQL réside dans la nature JSON des filtres MongoDB
3. Le pattern `safeFilter` est **efficace et nécessaire** – il ne suffit pas de “faire confiance” aux entrées utilisateur
4. La projection minimale constitue une **deuxième couche de défense** indépendante
5. Le mass assignment est une vulnérabilité silencieuse qui nécessite une allow-list explicite des champs modifiables

9.1 Checklist de sécurité retenue

Valider le type de chaque entrée (`typeof`)

Ne jamais faire `find(req.body)`

Appliquer une allow-list de valeurs

Valider les formats par regex

Filtrer les clés \$

Projection minimale sur tous les `find()`

Ne jamais exposer `err.message`

Jamais `$set:req.body` – allow-list des champs modifiables